

Lab 02 — Gradient-Based and Blind Optimization

Fundamentos de Aprendizagem Automática

Mário Antunes

2026

Overview

Class 02 · Topic 1. Lecture: `slides/02_optimization.pdf`. Demo notebooks in `notebooks/02-optimization/`: `01_gradient_based.ipynb` and `02_blind_optimization.ipynb`.

Notebook for this lab: `notebooks/02-optimization/lab02_optimization.ipynb`, with helpers in `lab02_utils.py` (same folder, do not edit). Cells marked **TODO** contain `raise NotImplementedError`: replace it with your code. Every other cell is ready, and it draws the paths, populations and statistics once your functions work.

Solution: `solutions/lab02_optimization_solution.ipynb` is the fully solved notebook. Before each function it explains how the solution was reached, and after each experiment it answers this guide's questions with the numbers the run produces. Try every task yourself first, and use the solution to check your reasoning or to get unstuck.

By the end of the lab you should be able to:

1. Derive a gradient by hand, check it with JAX, and run gradient descent with a sensible learning rate.
2. Design and implement a real-valued genetic algorithm, justifying every operator.
3. Use `pyBlindOpt`'s initialization methods and optimizers, and compare stochastic optimizers fairly.
4. Implement Newton's method with `jax.hessian`, and explain when it wins and when it fails.

Part	Task	Time
A	Gradient descent on P1–P3	30 min
B	Genetic algorithm from scratch	40 min
C	pyBlindOpt: initialization and stronger optimizers	25 min
D	Newton’s method	15 min
E	Challenge: optimizing the 0/1 loss	home

Setup

```
cd faa
```

```
source venv/bin/activate          # see README: make venv, or pip install .
jupyter lab notebooks/02-optimization/lab02_optimization.ipynb
```

The Three Problems

All three have two parameters, so every path and population can be drawn on the loss landscape.

	Loss $\mathcal{L}(\theta)$	Parameters	Character
P1	$\frac{1}{N} \sum_i (wx_i + b - y_i)^2$	$(w, b) \in [-5, 5]^2$	convex, elongated valley
P2	$(1 - x)^2 + 100(y - x^2)^2$	$(x, y) \in [-2, 2] \times [-1, 3]$	curved, badly conditioned
P3	$\frac{1}{N} \sum_i (\sin(\omega x_i + \varphi) - y_i)^2$	$(\omega, \varphi) \in [0.1, 4] \times [-\pi, \pi]$	multimodal

- **P1** fits a line to 40 noisy points. It is the simplest ML model, and its minimum has a closed form (least squares).
- **P2** is the Rosenbrock function: the minimum $(1, 1)$ lies at the bottom of a narrow, curved valley.
- **P3** fits the frequency and phase of a sinusoid to 80 noisy points. A wrong frequency can still match part of the data, which creates many local minima along ω .

In the notebook, `PROBLEMS["P1"]` has `.loss` (written with `jax.numpy`), `.numpy_loss` (the same function on NumPy arrays), `.bounds`, `.start`, `.optimum` and `.data`. Every loss takes `theta[... , 0]` and `theta[... , 1]`, so it

evaluates one point, shape $(2,)$, or a whole population, shape $(n, 2)$, in one call.

Part A — Gradient Descent

A1. The gradient of P1 by hand

Write the residual $r_i = wx_i + b - y_i$, so $\mathcal{L} = \frac{1}{N} \sum_i r_i^2$. Use the chain rule: $\partial r_i / \partial w = x_i$ and $\partial r_i / \partial b = 1$.

1. Derive $\partial \mathcal{L} / \partial w$ and $\partial \mathcal{L} / \partial b$.
2. Implement `grad_p1_manual(theta)` with NumPy and check it against `jax.grad` (the next cell prints both).
3. Set the gradient to zero. You get two linear equations, the *normal equations*. Why can this problem be solved without iterating at all?

A2. Gradient descent

$$\theta_{k+1} = \theta_k - \eta \nabla \mathcal{L}(\theta_k)$$

Implement `gradient_descent(grad, theta0, lr, n_steps)`. It returns the whole path, an array of shape $(n_steps + 1, 2)$. The path is what the plots draw.

A3. Learning rates

The driver cell runs three learning rates per problem and plots the paths and the loss curves.

1. **P1:** why does GD zig-zag across the valley instead of heading straight to the minimum? The Hessian of P1 is constant, with eigenvalues of about 0.36 and 8.2. Plain GD is stable only for $\eta < 2/\lambda_{\max}$. Check the prediction against the run with $\eta = 0.25$.
2. **P2:** find the largest learning rate that does not diverge. How many steps does it need to get within 10^{-3} of the minimum?
3. **P3:** where does GD end from the start point $(1, 0)$? Look at the fit plot: what did the model learn?

A4. Multi-start

Implement `multistart_gd`: run GD from `n_starts` uniform random points in the box and return the final points. What fraction reaches the global minimum of P3? How many gradient evaluations did that cost?

Part B — A Genetic Algorithm from Scratch

A genetic algorithm (GA) evolves a **population** of candidate solutions. Better candidates are more likely to become parents, parents are **recombined**, children are **mutated**, and the process repeats. It uses only loss *values*, never gradients.

Design considerations

Every choice below is a trade-off between **exploration** (looking at new regions) and **exploitation** (refining the good regions already found). Write down your choice and the reason for it.

Representation. The parameters are real numbers, so an individual is simply the vector $\theta \in \mathbb{R}^d$ (real-coded GA). Binary encodings are historical; for continuous parameters they add discretization error and make mutation step sizes awkward to control.

Fitness. The fitness of an individual is its loss (lower is better). Evaluate the whole population in one call: `loss(pop)` returns one value per row. Count evaluations. The budget of a blind optimizer is measured in **function evaluations**, not in generations.

Population size N . A larger N covers the space better but spends more evaluations per generation. For 2 parameters, $N = 20\text{--}50$ is plenty. With a fixed budget B , you choose between N large with few generations, and N small with many generations.

Initialization. Uniform sampling inside the box (`rng.uniform(low, high, (N, d))`). In Part C you will compare it with smarter samplers.

Selection: tournament of size k . Pick k individuals at random and keep the best. This controls the **selection pressure**:

- $k = 1$ is random selection: no pressure, so the GA is a random walk;
- $k = 2\text{--}3$ is mild pressure: the usual default;

- large k is strong pressure: fast convergence, but the population loses diversity and can collapse into a local minimum.

A tournament only compares losses. It does not care about their scale, so it works equally well for losses of order 10^{-3} or 10^4 (unlike fitness-proportional “roulette” selection).

Crossover: blend (BLX- α). For every gene j of parents p_1, p_2 , with $\ell_j = \min(p_{1j}, p_{2j})$, $u_j = \max(p_{1j}, p_{2j})$ and $d_j = u_j - \ell_j$:

$$c_j \sim \mathcal{U}(\ell_j - \alpha d_j, u_j + \alpha d_j)$$

With $\alpha = 0$ children stay *between* the parents, and the population can only shrink. With $\alpha = 0.5$ children may land slightly *outside* the parents, which preserves diversity. Produce two children per pair of parents.

Mutation: Gaussian. With probability rate per gene, add $\mathcal{N}(0, \sigma^2)$ with σ a fraction of the box width ($\sigma = 0.1 (u - \ell)$ by default). Mutation is the only operator that can reach values no parent has. If σ is too small, the population cannot leave a basin; if it is too large, the GA degrades into random search. With $d = 2$, a rate of 0.5 mutates about one gene per child. In higher dimensions the common rule is $1/d$.

Bounds. Children of BLX- α and mutation can leave the box. **Clip** them back (`np.clip`). Reflecting them back inside is an alternative.

Elitism. Copy the best `n_elite` individuals unchanged into the next generation. The best solution can then never be lost, so the best loss never gets worse from one generation to the next. Keep it small (1–2) to avoid premature convergence.

Termination. A fixed number of generations (a fixed budget) keeps runs comparable. Alternatives are a target loss or “no improvement in p generations” (patience).

Randomness. A GA is stochastic. Pass one `numpy.random.Generator` to every operator, so each run is reproducible from its seed. Never judge a configuration from a single run: compare **medians over many seeds** (Class 01, evaluation).

B1. Implement

`init_population`, `tournament`, `blend_crossover`, `gaussian_mutation`, then `genetic_algorithm`, which returns (`best_theta`, `best_loss`, `history`). Each generation:

1. sort by loss and copy the `n_elite` best;
2. until the new population is full: two tournaments $\rightarrow$ BLX- α $\rightarrow$ mutate and clip each child;
3. evaluate the new population and update the best individual ever seen.

B2. Run and compare

The driver runs your GA on P1–P3 (30 individuals, 60 generations, i.e. 1830 evaluations) and shows the population at generations 0, 3, 10 and 60.

1. On P1, compare the GA's final loss with gradient descent's. Which is more precise, and which is cheaper?
2. On P3, compare the fitted curves of GD and GA. Why does the population method find the right frequency?

B3. Parameters

The driver runs your GA 10 times for each setting, with a small budget (15 generations) so the differences show. Explain the boxes using the design considerations: $k = 1$ against $k = 8$, $\sigma = 0.01$ against $\sigma = 0.3$, and elitism against no elitism.

Part C — pyBlindOpt

C1. Initialization methods

Implement `make_population(kind, problem, n_pop, rng)` for:

- **samplers** (`pyBlindOpt.utils`): `RandomSampler`, `HLCSSampler` (Latin hypercube), `SobolSampler`, `ChaoticSampler`. Use them with `init.get_initial_population(bounds, sampler)`;
- **strategies** (`pyBlindOpt.init`): `opposition_based`, `quasi_opposition_based`, `oblesa`. Pass `population=utils.RandomSampler(rng)`, `n_pop` and `seed=rng`.

Samplers only place points; strategies also *evaluate* candidates and keep the best ones. Which methods spend evaluations before the search starts? The plot shows the best initial loss on P3. Why do QOBL and OBLESA start so much better? (Hint: where is the centre of the box, and where is the optimum?)

C2. Stronger optimizers

Implement `run_library(name, problem, population, n_iter, seed)` for "GA", "DE", "SHADE" and "EGWO":

```
pbo.differential_evolution(f, bounds, population=pop, n_iter=n_iter, seed=seed)
pbo.differential_evolution(f, bounds, variant="current-to-pbest/1/bin", p=1, seed=seed)
pbo.enhanced_grey_wolf_optimization(f, bounds, ...)
pbo.genetic_algorithm(f, bounds, ...)
```

Each call returns `(best_theta, best_loss)`. The drivers compare the optimizers, and then DE with every initialization method, with the **same budget** (620 evaluations) and 15 seeds. Answer:

1. Which optimizer is best on each problem? Is any one best on all three?
2. How does your GA compare with pyBlindOpt's GA with the same budget?
3. Does the initialization change the result of DE? On which problem does it matter most?

C3. Watch one run

The class interface (`pbo.DifferentialEvolution(...)`) keeps the optimizer object. A `Recorder` callback stores the population at every epoch. Compare DE's population snapshots with your GA's: how does each one contract onto the optimum?

Part D — Newton's Method

$$\theta_{k+1} = \theta_k - (H(\theta_k) + \lambda I)^{-1} \nabla \mathcal{L}(\theta_k), \quad H = \text{jax.hessian}(\mathcal{L})$$

Implement `newton(problem, theta0, n_steps, damping)`. Use `np.linalg.solve(H, -g)`, not an explicit inverse. The driver compares 20 steps of GD, pure Newton ($\lambda = 0$) and damped Newton ($\lambda = 1$) on each problem.

1. P1 is a quadratic. Why does Newton finish in exactly one step?
2. P2: how many steps does Newton need, compared with GD in Part A?
3. P3: where does pure Newton go from $(1, 0)$? Compute the eigenvalues of the Hessian at its final point (`np.linalg.eigvalsh`). What kind of point is it?

4. With d parameters the Hessian has d^2 entries and the solve costs $O(d^3)$. Why is Newton not used to train neural networks with millions of weights?

Part E — Challenge: Optimizing What We Care About

A linear classifier predicts $\hat{y} = [w^\top x + b > 0]$. We judge it by its **error rate** (the 0/1 loss), but that loss is **piecewise constant**. A small change of θ usually flips no prediction, so the gradient is zero almost everywhere.

The data: 300 training and 1000 test points, 6 features, 10% of the labels flipped at random (make_classification). The parameters are $\theta = (w_1, \dots, w_6, b) \in [-5, 5]^7$.

1. Implement `error_rate(theta, X, y)` for one θ (shape $(7,)$) **and** for a population (shape $(n, 7)$), vectorized with one matrix product.
2. Implement `logistic_loss(theta, X, y)` with `jax.numpy`:

$$\mathcal{L}_{\log}(\theta) = \frac{1}{N} \sum_i \left[\log(1 + e^{z_i}) - y_i z_i \right], \quad z_i = w^\top x_i + b$$

Use `jnp.logaddexp(0, z)` for $\log(1 + e^z)$: it does not overflow.

3. The driver prints `jax.grad` of the 0/1 loss. Explain the result.
4. Compare the test errors of GD on the logistic loss and of DE, SHADE, EGWO and your GA on the 0/1 loss itself (10 seeds, 6040 evaluations per run). Which is lower, which is more stable, and which is cheaper?
5. Why does multiplying θ by a positive constant not change the error rate? What does that do to the landscape the blind optimizer sees?

Going further. Replace the error rate with a metric that has no useful gradient either: the F1 score, or balanced accuracy on imbalanced classes. Or tune the learning rate and the number of steps of gradient descent with a blind optimizer (hyperparameter optimization).